DECISION TREES

```
> options(digits=2)
> library("rpart")
> load("/Users/shaina/Desktop/Data Sets/PenDigits/PenDigits.Rdata")
> attach(train)
> tree = rpart(digit~. - digit, data=train, method='class',
+              parms = list(split='entropy'))
```

To display the decision tree (and then label it), use the plot() and text() functions. The option uniform=T causes all the branches of the tree to have the same length. The first two commands and the last command in this chunk of code are merely used to set and reset the margins of the plot window.

```
> .pardefault = par()
> par(mai=c(.2,.2,.2,.2))
> plot(tree, uniform=T)
> #text(tree, use.n=T)
> par(.pardefault)
```

All we see in the resulting tree (Figure 1) is the *decision* at every leaf. While this tells us the most prevalent class of each leaf, it does not get at how accurate that prediction is, or what the predicted probability actually is. We can get that information by adding the option use.n=T to the text() function, but it's not particularly nice in our current 10-class prediction problem.

Unfortunately the tree does not even tell us *how* the split should be used. For example, the first split in the tree is '$V14 \geq 52.5$.' However, the tree does not indicate whether observations meeting that constraint should proceed down the left or right branches of the tree! I'll save you some trouble and let you know that the 'TRUE' values, i.e. those that meet the given condition, are sent down the LEFT branch of the tree.

Variable Importance

We can also obtain statistics describing variable importance. In general, variable importance is typically calculated using some impurity function, $i(\dot{)}$ that measures the impurity of a given node like *entropy* or the *Gini coefficient*. Recall that we decide to split some node if the change in that impurity measure is significant. If t_L is the left child of a split and t_R is the right child of a given node t, and N_L, N_R, N are the number of observations in the left, right and root node respectively, then the change in that impurity measure is calculated as

$$\Delta i(t) = i(t) - \frac{N_L}{N} i(t_L) - \frac{N_R}{N} i(t_R).$$

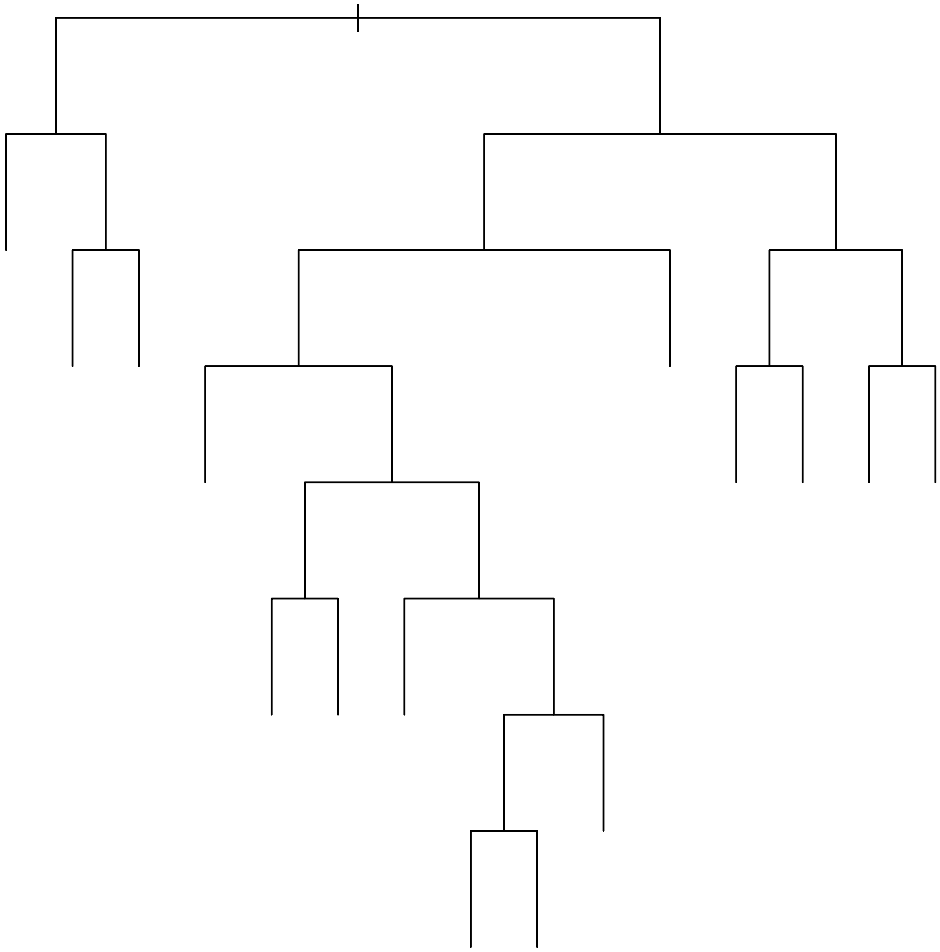


Figure 1: Decision Tree for PenDigits Data

We use that change in impurity measure then to calculate the **variable importance** of variable $\mathbf{x}_k$, denoted $I(\mathbf{x}_k)$ by adding up the weighted impurity decreases for all nodes t where $\mathbf{x}_k$ is used. If $\mathbf{x}_k$ is only used in one split, then the variable importance is just the impurity decrease for that one split. (Note: if using random forests, we can average this importance over all trees in the forest.)

$$I(\mathbf{x}_k) = \sum_t \frac{N_t}{N} \Delta i(t)$$

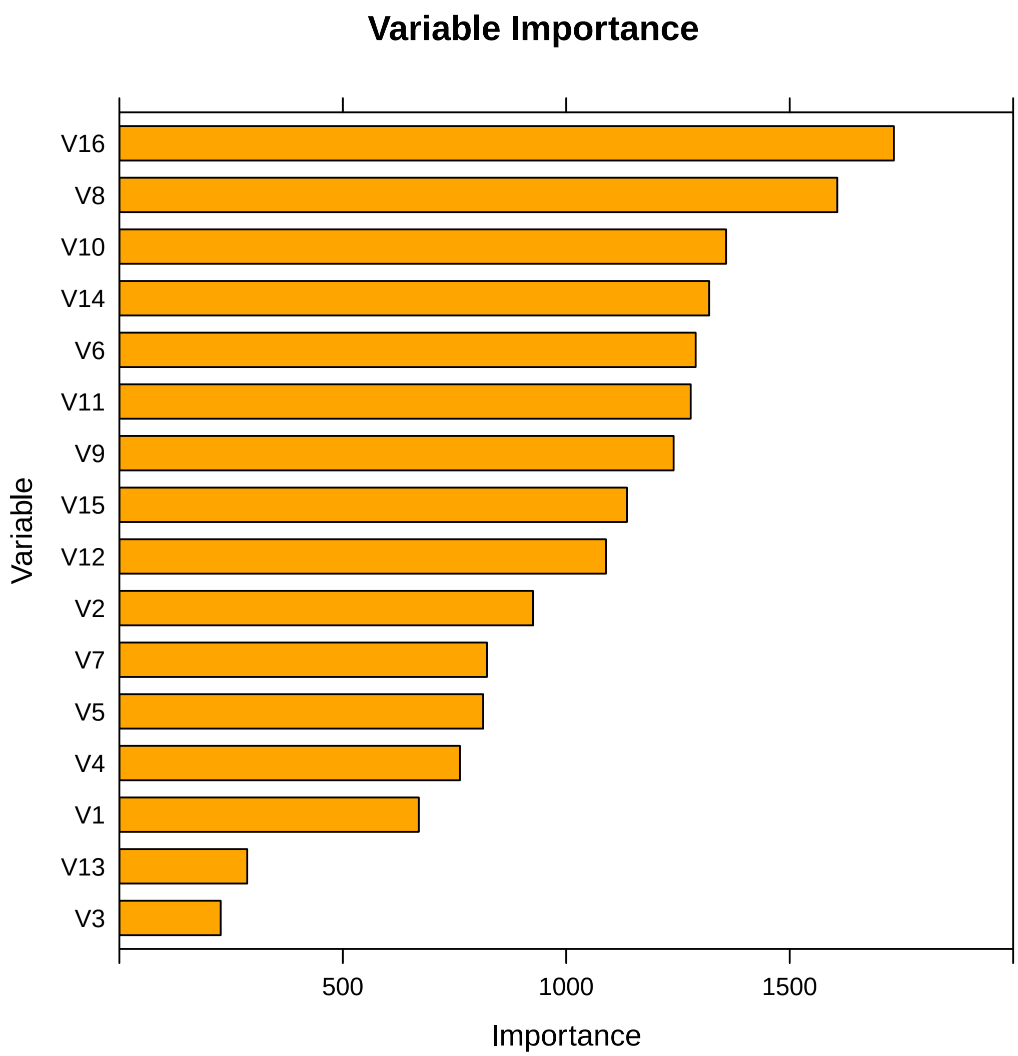
For our PenDigits dataset, we can examine the variable importance measures using the following code:

```
> tree$variable.importance
```

```
 V16   V8  V10  V14   V6  V11   V9  V15  V12   V2   V7   V5   V4   V1  V13   V3
1733 1607 1358 1320 1290 1278 1240 1136 1089  926  822  814  762  670  286  227
```

Or create a bar plot to quickly investigate the relative differences,

```
> library('lattice')
> barchart(tree$variable.importance[order(tree$variable.importance)],
+         xlab = 'Importance', horiz=T, xlim=c(0,2000),ylab='Variable',
+         main = 'Variable Importance',cex.names=0.8, las=2, col = 'orange')
```



Computing Validation Misclassification

R has a nice function, predict(), which allows us to apply a model to a new set of data. The resulting output is just a vector or matrix of predictions. In this case, we'll have an $n \times 10$ matrix of predicted probabilites because for each observation we predict the probability that it belongs to each possible class (or that the observation is each digit 0 through 9). We'll have to work with those predictions to determine misclassification rate ourselves. Let's compute training misclassification while we're at it.

```
> tscores = predict(tree,type='class')
> scores = predict(tree, test, type='class')
> cat('Training Misclassification Rate:',
+     sum(tscores!=train$digit)/nrow(train))
```

```
Training Misclassification Rate: 0.16
```

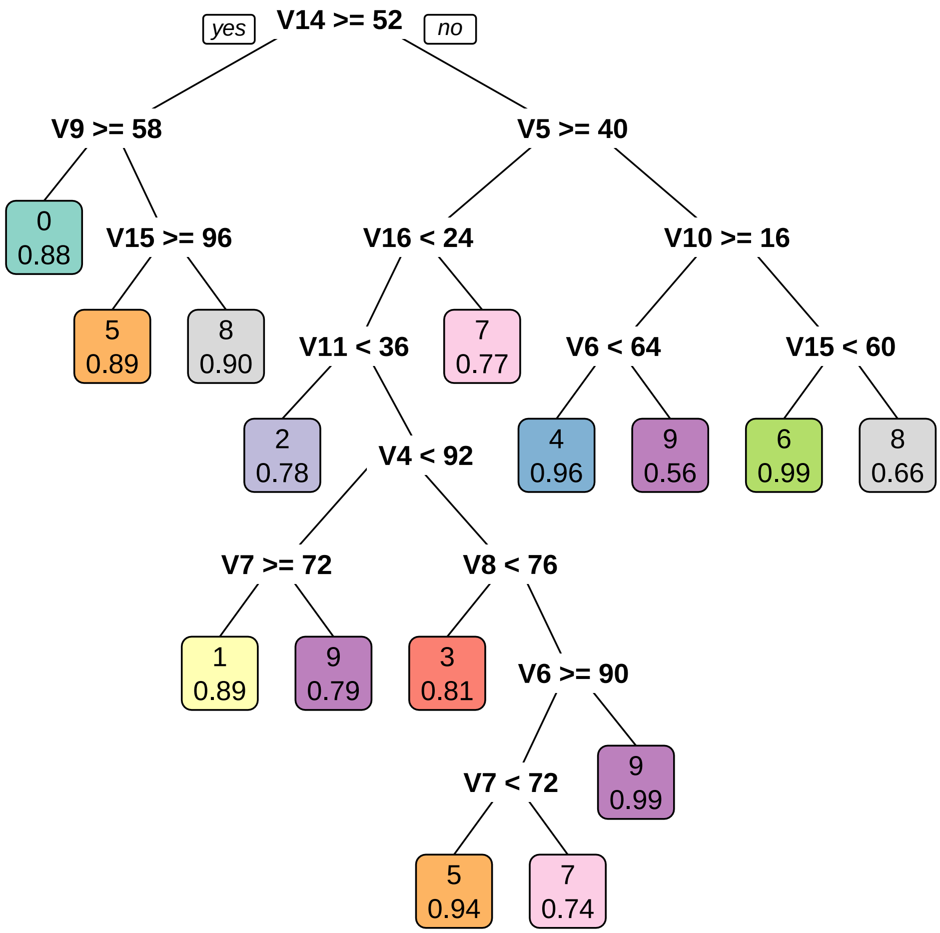
```
> cat('Validation Misclassification Rate:',
+     sum(scores!=test$digit)/nrow(test))
```

```
Validation Misclassification Rate: 0.26
```

Visual Aesthetics

As you've already noticed, the tree plots straight out of the rpart package leave much to be desired. Luckily others have put some work into giving us more plot options. Check out the prp() function in the rpart.plot library. Below we look at some quick examples of how these plots can look a little more appealing.

```
> library("rattle")                                    # Fancy tree plot
> library("rpart.plot")                          # Enhanced tree plots
> library("RColorBrewer")                  # Color selection for fancy tree plot
> library("party")                                          # Alternative decision tree algorithm
> library("partykit")                                  # Convert rpart object to BinaryTree
> # fancyRpartPlot(tree)      # Looks completely terrible but has
> #                           # potential for smaller trees, fewer classes
> #
> # prp(tree)
> # prp(tree, type =3, extra=100) # label branches, label nodes with % of obs
> # prp(tree, type =3, extra=2) # label branches, label nodes with misclass rate
> # prp(tree, type =3, extra=8) # label branches, label nodes with pred prob of class
> # # BEWARE WITH BINARY TREES WHERE WE TYPICALLY WANT TO SHOW PROB OF SUCCESS/FAILURE
> # # FOR EVERY NODE IN THE TREE!
> prp(tree, type =0, extra=8, leaf.round=1, border.col=1,
+     box.col=brewer.pal(10,"Set3")[tree$frame$yval], )
```


yes
V14 >= 52
no
V9 >= 58
V5 >= 40
0
0.88
V15 >= 96
V16 < 24
V10 >= 16
5
0.89
8
0.90
V11 < 36
7
0.77
V6 < 64
V15 < 60
2
0.78
V4 < 92
4
0.96
9
0.56
6
0.99
8
0.66
V7 >= 72
V8 < 76
1
0.89
9
0.79
3
0.81
V6 >= 90
V7 < 72
9
0.99
5
0.94
7
0.74